

```

LeetCode 42 int maxArea (int* height, int heightSize) {
    int left, right;
    int area = 0;
    int ar2 = 0;
    right = heightSize - 1; left = 0;
    while (left <= right) {
        if (height[left] > height[right]) {
            ar2 = (right - left) * height[right];
            right--;
        }
        else {
            ar2 = (right - left) * height[left];
            left++;
        }
        if (area < ar2) {
            area = ar2;
        }
    }
    return area;
}

```

Test case 1

o/p $\rightarrow$ height = [1, 8, 6, 2, 5, 4, 8, 3, 7]

49

Test case 2

height = [1, 1]

o/p 1

```

34 int * searchRange(int *nums, int numsSize,
                    int target, int *returnSize) {
    int n = *returnSize = 2;
    int *arr, j = 0;
    arr = (int *) (malloc(n * sizeof(int)));
    arr[0] = arr[1] = -1;
    for (int i = 0; i < numsSize; i++) {
        if (nums[i] == target) {
            arr[0] = i;
            j = i;
            break;
        }
    }
    for (int k = j; k < numsSize; k++) {
        if (nums[k] == target) {
            arr[1] = k;
        }
    }
    return arr;
}

```

Test case - 1;

nums = [5, 7, 7, 8, 8, 10] target = 8

o/p [3, 4]

Test case 2

nums = [5, 7, 7, 8, 8, 10] target = 6

o/p [-1, -1]

BFS

```

#include <stdio.h>
int a[10][10], vis[10], n;
void bfs(int);
void main() {
    int i, j, src;
    printf("Enter number of vertices");
    scanf("%d", &n);
    printf("Enter adjacency matrix");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &a[i][j]);
        }
        vis[i] = 0;
    }

    printf("Enter the source vertex");
    scanf("%d", &src);
    printf("Nodes reachable from src vertex");
    bfs(src);
}

void bfs(int v) {
    int q[10], f = 1, r = 1, u, i;
    q[r] = v;
    vis[v] = 1;
    while (f <= r) {
        u = q[f];
        printf("%d", u);
        for (i = 1; i <= n; i++) {
            if (a[u][i] == 1 && vis[i] == 0) {
                vis[i] = 1;
                r = r + 1;
                q[r] = i;
            }
        }
        f = f + 1;
    }
}

```


DFS

```
#include <stdio.h>
void dfs(int v);
int n, i, j, a[10][10], vis[10], count;
void main() {
    printf("Enter the number of vertices");
    scanf("%d", &n);
    printf("Enter adjacency matrix");
    for(i=1; i<=n; i++) {
        for(j=1; j<=n; j++) {
            printfscanf("%d", &a[i][j]);
        }
        vis[i] = 0;
    }
    printf("DFS traversal");
    for(i=1; i<=n; i++) {
        if(vis[i] == 0) dfs(i);
    }
    for(i=1; i<=n; i++) {
        if(vis[i])
            count++;
    }
    if(count == n) printf("Connected graph");
    else
        printf("Disconnected graph");
}

void dfs(int v) {
    vis[v] = 1;
    printf("%d", v);
    for(j=1; j<=n; j++) {
        if(a[v][j] == 1 & vis[j] == 0) {
            dfs(j);
        }
    }
}
```

o/p

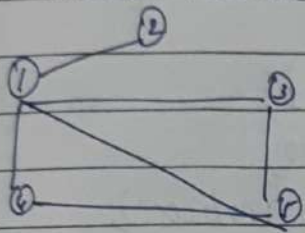
BFS:- Enter number of vertices: 5

Enter adjacency matrix

0	1	1	1	1
1	0	0	0	0
1	0	0	0	1
1	0	0	0	1
1	0	1	1	0

Enter source vertex: 1

Nodes reachable from 1: 1 2 3 4 5



DFS:- Enter number of vertices: 5

Enter adjacency matrix:

0	1	1	1	0
0	0	1	0	0
0	1	0	0	0
0	0	0	1	0
0	1	0	0	0

DFS Traversal: 1 2 3 ~~4~~ 5

Graph not connected

LAB PROGRAM-1

Date / /
Page

Q. WAP to obtain the topological sorting/ordering of vertices in a given digraph.

Code

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int main()
{
    int n, adj[MAX][MAX];
    int in[MAX] = {0};
    int q[MAX], f = 0, r = -1;
    int topo[MAX], count = 0;
    printf("Enter number of vertices");
    scanf("%d", &n);
    printf("Enter adjacency matrix");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &adj[i][j]);
        }
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (adj[i][j] == 1)
            {
                in[j]++;
            }
        }
    }
    for (int i = 0; i < n; i++)
    {
        if (in[i] == 0)
        {
            q[f++] = i;
        }
    }
    while (f <= r)
    {
        int v = q[f++];
        topo[count++] = v;
    }
}
```



```

for(int i=0; i<n; i++){
    if (adj[v][i]==1){
        in[i]--;
        if(in[i]==0){
            q[++r]=i;
        }
    }
}

if(count!=n){
    printf("Graph has a cycle. Topological ordering not possible");
} else {
    printf("Topological ordering");
    for(int i=0; i<n; i++){
        printf("%d", topo[i]);
    }
    return 0;
}

```

O/P

Enter number of vertices: 4
Enter adjacency matrix:

```

0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

```

Topological ordering: 0 1 2 3.

mq

96. Leetcode Problem on Topological Ordering [Leet #207. Course Schedule].

```

Code
bool canFinish(int numCourses, int** prerequisites,
               int prerequisitesSize, int* prerequisitesColSize) {
    int count = 0;
    int *in = (int *) calloc (numCourses, sizeof(int));
    int **gr = (int **) malloc (numCourses, sizeof(int));
    int *grSize = (int *) calloc (numCourses, sizeof(int));

    for (int i = 0; i < numCourses; i++) {
        gr[i] = (int *) malloc (prerequisitesSize * sizeof(int));
    }

    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        gr[b][grSize[b]++] = a;
        in[a]++;
    }

    int *q = (int *) malloc (numCourses * sizeof(int));
    int f = 0, r = 0;
    for (int i = 0; i < numCourses; i++) {
        if (in[i] == 0) {
            q[r++] = i;
        }
    }

    while (f < r) {
        int c = q[f++];
        count++;
        for (int i = 0; i < grSize[c]; i++) {
            int next = gr[c][i];
            in[next]--;
            if (in[next] == 0) {
                q[r++] = next;
            }
        }
    }

    return count == numCourses;
}

```


o/p Case 1
prerequisites = [[1,0]]
numCourses = 2
output = true

Case 2
numCourses = 2
prerequisites = [[1,0],[0,1]]
output = false

LEETCODE

Missing Number

```
268 int missingNumber(int *nums, int numsSize){
    int sum=0;
    for (int i=0; i<numsSize; i++){
        sum = sum + nums[i];
    }
    int sum2 = numsSize * (numsSize + 1) / 2;
    return sum2 - sum;
}
```

o/p Case 1 Case 2 Case 3
nums = [3,0,1] nums = [0,1] nums = [9,6,4,2,3,5,7,
out = 2 out = 2 out = 8
0,1]

LEETCODE

```
1676 int freq(int k, int *num, int numsSize){
    int c=0;
    for (int i=0; i<numsSize; i++){
        if (num[i]==k) c++;
    }
    return c;
}
```

```
int *frequencySort(int *nums, int numsSize,
    int *returnSize){
    *returnSize = numsSize;
    int *f = (int *) malloc(numsSize * sizeof(int));
    for (int i=0; i<numsSize; i++){
        f[i] = freq(nums[i], nums, numsSize);
    }
}
```

```

for (int j=0; j<numsSize; j++) {
    for (int k=0; k<numsSize; k++) {
        if ( (f[k]>f[k+1]) ||
              (f[k]==f[k+1] & &
               nums[k]<nums[k+1]) ) {
            int temp = nums[k];
            nums[k] = nums[k+1];
            nums[k+1] = temp;
            int temp2 = f[k];
            f[k] = f[k+1];
            f[k+1] = temp2;
        }
    }
}
return nums;
}

```

o/p

Case 1
 nums = [2, 3, 1, 3, 2]
 out = [1, 3, 3, 2, 2]

Case 2
 nums = [1, 1, 2, 2, 3]
 out = [3, 1, 1, 2, 2, 2]

NA
9/3

LAB PROGRAM-2 LAB-3

Ques 2. Implement Johnson - Trotter Algorithm to generate permutations.

Algorithm / Pseudocode

1. create array $permutation[n]$, $direction[n]$.
2. for $i = 1 - N$
 $permutation[i] = i$
 $direction[i] = LEFT$
3. print permutations
4. While TRUE
 $mobile = largest_mobile(permutation, direction)$
 if $mobile == 0$
 $BREAK / STOP$
 $pos = position\ of\ mobile$
 if $direction[mobile] == LEFT$
 $swap\ permutation[pos]\ and\ permutation[pos - 1]$
 else
 $swap\ permutation[pos]\ &\ permutation[pos + 1]$
 $direction[mobile] = reverse\ direction$
 for x in $permutation$
 if $x > mobile$
 $direction[x] = reverse\ direction$
 print permutations.

NG.

CODE:-

```
#include <stdio.h>
#define l -1
#define r -1

void PrinPerm (int perm[], int n) {
    for (int i=0; i<n; i++) {
        printf("%d ", perm[i]);
    }
    printf("\n");
}

int getMobile (int perm[], int dir[], int n) {
    int mobile = 0;
    for (int i=0; i<n; i++) {
        if (dir[perm[i]-1] == l && i!=0) {
            if (perm[i] > perm[i-1] && perm[i] > mobile)
                mobile = perm[i];
        }
        if (perm[i] > perm[i+1] && perm[i] > mobile)
            mobile = perm[i];
    }
    return mobile;
}

int search (int perm[], int n, int mobile) {
    for (int i=0; i<n; i++) {
        if (perm[i] == mobile)
            return i;
    }
}

void johnsonTrotter (int n) {
    int perm[n], dir[n];
    for (int i=0; i<n; i++) {
        perm[i] = i+1; dir[i] = l;
    }
}
```

```
print Perm (perm, n);
```

```
while (1){
```

```
    int mobile = getMobile (perm, dir, n);
```

```
    if (mobile == 0)
```

```
        break;
```

```
    int pos = search (perm, n, mobile);
```

```
    if (dir[mobile-1] == 1){
```

```
        int temp = perm[pos];
```

```
        perm[pos] = perm[pos-1];
```

```
        perm[pos+1] = temp;
```

```
    }
```

```
    else{
```

```
        int temp = perm[pos];
```

```
        perm[pos] = perm[pos+1];
```

```
        perm[pos+1] = temp;
```

```
    }
```

```
    for (int i=0; i<n; i++){
```

```
        if (perm[i] == mobile){
```

```
            dir[perm[i]-1] = -dir[perm[i]-1];
```

```
        }
```

```
        print Perm (perm, n);
```

```
    }
```

```
}
```

```
int main () {
```

```
    int n;
```

```
    printf ("Enter n: ");
```

```
    scanf ("%d", &n);
```

```
    johnsonTrotter (n);
```

```
    return 0;
```

```
}
```

o/p

Enter n: 3

1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Time Complexity

$$T(n) = nT(n-1) + O(n) = nT(n-1) + Cn$$

Put $n-1$ in n

$$T(n-1) = nT(n-2) + O(n-1)$$

$$T(n) = \cancel{nT(n-1)} - n(n-1)T(n-2) + C(n+1)$$

$$\cancel{T(n)} T(n) = n(n-1)(n-2) \dots (n-k+1) + C(n(n-1)(n-k+1) \dots + n)$$

$$k = n-1, \quad T(n) = n! + (1) + C(n! + \dots)$$

$$T(1) = C$$

$$\underline{\underline{T(n) = O(n!)}}$$

mg
16/3


```

1/79 #include <stdio.h>
int gcd(int, int);
void main()
{
    int a, b;
    printf("Enter the numbers"); int res = gcd(a, b);
    scanf("GCD is %d", res);
}

int gcd(int a, int b)
{
    if (b == 0)
        return a;
    return gcd(b, a % b);
}

```

o/p Enter the numbers: 4 3
GCD is 1

Time Complexity - $O(\log(\min(a, b)))$

```

2/76 #include <stdio.h>
int fib(int);
void main()
{
    int n;
    printf("Enter a number");
    scanf("%d", &n);
    int res = fib(n);
    printf("%d", res);
}

int fib(int n)
{
    if (n <= 1)
        return 1;
    return fib(n-1) + fib(n-2);
}

```

Time Complexity : $O(2^n)$

Recurrence rel : $T(n) = T(n-1) + T(n-2)$

$n = n-1 \Rightarrow T(n-1) = T(n-2) + T(n-3)$

$T(n) = 2T(n-2) + T(n-3)$

$T(n) = 2(k-1)T(n-k+1) + T(n-k)$
 $\approx 2^n$

Tower of Hanoi

3.

#include <stdio.h>

void toh (int n, char ^a~~A~~, char ^c~~B~~, char ^b~~C~~) {

if (n==1) {

printf("move disc 1 from %c to %c\n", a, b);
return;

}

toh(n-1, a, b, c);

printf("Move disc %d from %c to %c", n, a, b);

toh(n-1, c, a, b);

}

int main() {

int n;

printf("Enter number of discs");

scanf("%d", &n);

toh(n, 'A', 'C', 'B');

return 0;

}

Time Complexity :- $O(2^n)$

Recurrence relation :- $T(n) = 2T(n-1) + 1$

$n = n-1 \Rightarrow T(n-1) = 2T(n-2) + 1$

$T(n) = 4T(n-2) + 2 + 1$

$= 2^n T(n-n) + n + (n-1) + \dots + 1$

$= 2^n + \frac{n(n+1)}{2}$

$\approx O(2^n)$

Selection Sort

```
> #include <stdio.h>
void selectionSort(int arr[], n)
{
    int i, j, min, temp;
    for (i=0; i<n; i++) {
        min=i;
        for (j=i+1; j<n; j++) {
            if (arr[j]<arr[min]) {
                min=j;
            }
        }
        temp = arr[min];
        arr[min] = arr[i];
        arr[i] = temp;
    }
}
```

```
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
    selectionSort(arr, n);
    for (int i=0; i<n; i++) {
        printf("%d", arr[i]);
    }
    return 0;
}
```

o/p 11 12 22 25 64

Time Complexity = $O(n^2)$

Recurrence relation $\Rightarrow T(n) = T(n-1) + (n-1)$

$n-1 \Rightarrow T(n-1) = T(n-2) + (n-2)$

$T(n) = T(n-2) + (n-1) + (n-2)$

$\vdots$

$T(n) = T(n-n) + 1 + 2 + \dots + (n-1)$

$= \frac{n(n-1)}{2} \approx n^2$

$O(n^2)$

Best case = Worst case = Average case = n^2

Bubble Sort

```
5) #include <stdio.h>
void main()
{
    int arr[10] = {5, 2, 9, 1, 5, 6, 7, 3, 4, 8};
    for (int i = 0; i < 10; i++)
    {
        for (j = i+1; j < 10; j++)
        {
            if (arr[i] > arr[j])
            {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
    for (int i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

o/p 1 2 3 4 5 5 6 7 8 9

$$\text{Recurrence} = T(n) = T(n-1) + (n-1)$$

∴ same as selection sort

∴

Time Complexity = $O(n^2)$.

PROGRAM-3 ⇒ Sort a given set of n integers.

using merge Sort and compute its time taken.
Run the program for different values of n and record time taken to Sort.

ODE

```
#include <stdio.h>

void merge(int arr[], int aarr[], int al, int b, int bl);
void mergeSort(int arr[], int n);

int main();
{
    int n; arr[n]
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    for printf("Enter elements");
    for (int i=0; i<n; i++) {
        scanf("%d", &arr[i]);
    }
    mergeSort(arr, n); printf("Sorted array:");
    for (int i=0; i<n; i++) {
        printf("%d", arr[i]);
    }
    return 0;
}

void mergeSort(int arr[], int n);
{
    if (n <= 1) return;
    int al = n/2;
    int bl = n - n/2;
    int a[al] b[bl];
    for (int i=0; i<al; i++) {
        a[i] = arr[i];
    }
    for (int j=0; j<bl; j++) {
        b[j] = arr[i];
    }
}
```

```

mergeSort(a, al),
mergeSort(b, bl);
merge(arr, a, al, b, bl);
void merge(int arr[], int al, int al, int bl, int bl)
{
    int i=a, j=b, k=0;
    while (i < al && j < bl) {
        if (a[i] < b[j]) {
            arr[k++] = a[i++];
        } else {
            arr[k++] = b[j++];
        }
    }
    while (i < al) {
        arr[k++] = a[i++];
    }
    while (j < bl) {
        arr[k++] = b[j++];
    }
}

```

Algorithm

1. if $n \leq 1$ return
2. Divide array into 2 halves
one of size $n/2$, other of size $n-n/2$
3. Copy elements into 2 subarrays a, b
4. Recursively call:
 $\text{mergeSort}(a, \text{size of } a)$
 $\text{mergeSort}(b, \text{size of } b)$
5. Merge Sorted subarrays
6. End

Time Complexity $\Rightarrow T(n) = O(n \log n)$

$$T(n) = \begin{cases} 1, & \text{if } n=1 \\ 2T(n/2) + n, & \text{else} \end{cases}$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$n = \frac{n}{2} \quad \nearrow \quad T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \frac{n}{2}$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 2n$$

$$T(n) = 8T\left(\frac{n}{8}\right) + 3n$$

$$T(n) = \frac{n^k}{2^k} T\left(\frac{n}{2^k}\right) + \text{some const}$$

$$= n \cdot T(1) + n \log_2 n + \dots$$

$$= n + n \log_2 n$$

$$\approx \underline{\underline{n \log n}}$$

~~o/p~~ ①

Enter number of elements: 5

Enter elements: 43 23 78 99 55

Sorted array 23 43 55 78 99

execution time: 10.7275.

②

Enter number of elements: 10

Enter elements: 34 23 78 65 43 1 23 776 54 3

Sorted array 1 3 23 34 43 54 65 78 776

execution time: 19.648 s.

LAB program $\Rightarrow$ Sort a given set of n integers.

using Quick Sort technique and compute time taken.

Code: #include <stdio.h>

```
int main void quickSort (int a[], int low, int high);  
int partition (int a[], int low, int high);  
int main() {
```

```
    int n;  
    printf ("Enter number of elements");  
    scanf ("%d", &n); int arr[n];  
    printf ("Enter elements");  
    for (int i=0; i<n; i++) {  
        scanf ("%d", &arr[i]);  
    }
```

```
    quickSort (arr, 0, n-1); printf ("Sorted array\n");  
    for (int i=0; i<n; i++) {  
        printf ("%d ", arr[i]);  
    }  
    return 0;
```

```
}  
void quickSort (int a[], int low, int high) {  
    int j;  
    if (low < high) {  
        j = partition (a, low, high);  
        quickSort (a, low, j-1);  
        quickSort (a, j+1, high);  
    }  
}
```

```
int partition (int a[], int low, int high) {  
    int i, j, key, temp;  
    i = low + 1; j = high;  
    key = a[low];  
    while (1) {  
        while (a[i] <= key && i < high) {
```

```

        i++;
    while (a[j] > key) { j++; }
    if (i < j) {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
    else {
        temp = a[j];
        a[j] = a[low];
        a[low] = temp;
        return j;
    }
}

```

- Algorithm
1. If $low \geq high$, return // Quick sort
 2. Partition array using j .
 elements $< arr[j]$ go left
 rest go right
 3. Recursively call
 quicksort ($arr, low, j-1$)
 quicksort ($arr, j+1, high$)
 4. End
1. Choose key // partition
 2. Initialize $i = low + 1, j = high$
 3. While (1)
 if $arr[i] < key$
 i++
 swap $arr[i]$ & $arr[j]$
 4. Swap ~~$arr[i]$ & $arr[j]$~~ $arr[i+1]$ & $arr[low]$ low
 5. Return i ~~j~~;

Time Complexity :- Best Case $T(n) = O(2T(\frac{n}{2}) + n)$
 Avg Case $= O(n \log n)$
 Worst Case $= O(n^2)$

Recurrence $T(n) = 2T(\frac{n}{2}) + n$

MG
23/3

o/p: Enter number of elements: 5
 Enter elements: 12 43 66 43 21
 Sorted array 12 21 43 43 66
 Execution time: 8.408

Enter number of elements: 7
 Enter elements: 3 65 434 265 65 3 365
 Sorted array 3 3 65 65 265 365 434
 Execution time: 9.4936.

MG

1. Counting Sort

```

#include <stdio.h>
void count_sort(int arr[], int n) {
    int max = -1;
    for (int i = 0; i < n; i++) { if (arr[i] > max) {
                                    max = arr[i]; } }
    for (int i = 0; i < max + 1; i++) {
        count[i] = 0; }
    for (int i = 0; i < n; i++) {
        count[arr[i]]++; }
    int k = 0;
    for (int j = 0; j < max + 1; j++) {
        while (count[j] != 0) {
            arr[k] = j;
            count[j]--; k++; } }
}

void main() { int n;
    printf("Enter number of elements");
    scanf("%d", &n); int arr[n];
    printf("Enter elements");
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    count_sort(arr, n); printf("Sorted array:");
    for (int i = 0; i < n; i++) printf("%d", arr[i]);
}

```

o/p

Enter number of elements: 5
 Enter elements: 6 5 5 3 45
 Sorted array: 3 5 5 6 45

Time Complexity: $O(n+k)$ $\rightarrow$ max val

2. Shell Sort

```
#include <stdio.h>

void shellSort(int arr[], int n){
    for(int gap = n/2; gap > 0; gap /= 2){
        for(int i = gap; i < n; i++){
            int temp = arr[i], j;
            for(j = i; j >= gap && arr[j-gap] > temp; j -= gap)
                arr[j] = arr[j-gap];
            arr[j] = temp;
        }
    }
}
```

```
int main(){
    int arr[] = {12, 34, 54, 2, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    shellSort(arr, n);
    for(int i = 0; i < n; i++) printf("%d ", arr[i]);
}
```

Q. o/p 2 3 12 34 54

Time Complex = $O(n \log n)$
Avg = $O(n^2)$

3. Bucket Sort

Worst $O(n^2)$

```
#include <stdio.h>
#include <stdlib.h>

void insertionSort(float arr[], int n){
    for(int i = 1; i < n; i++){
        int key = arr[i], j = i - 1;
        while(j >= 0 && arr[j] > key){
            arr[j+1] = arr[j]; j++;
        }
        arr[j+1] = key;
    }
}
```

```
void bucketSort(int arr[], int n){
    int max = arr[0];
    for(int i = 1; i < n; i++) if(arr[i] > max)
        max = arr[i];
}
```



```

    int bucketCount = max/10+1;
    int *buckets[bucketCount], count[bucketCount];
    for (int i=0; i<bucketCount; i++){
        buckets[i] = (int *)malloc(n*sizeof(int));
        count[i] = 0;
    }
    for (int i=0; i<n; i++){
        int b = arr[i]/10;
        buckets[b][count[b]++] = arr[i];
    }
    int id=0;
    for (int i=0; i<bucketCount; i++){
        insertionSort(buckets[i], count[i]);
        for (int j=0; j<count[i]; j++){
            arr[id++] = buckets[i][j];
            free(buckets[i]);
        }
    }
    int main(){
        int arr = {42, 32, 33, 52, 37, 47, 51};
        int n = sizeof(arr)/sizeof(arr[0]);
        bucketSort(arr, n);
        for (int i=0; i<n; i++) printf(arr[i]);
        return 0;
    }

```

o/p

32 33 37 42 47 51 52

Time Complexity: $O(n+k) \rightarrow$ Best, Avg; $O(n^2)$ Worst

4. Radix Sort

Date / /
Page

```
#include <stdio.h>

int getMax(int arr[], int n) { int max = arr[0];
    for(int i=1; i<n; i++) { if (arr[i] > max)
        max = arr[i]; }
    return max;
}

void countingSort(int arr[], int n, int exp) {
    int o[n], count[10] = {0};
    for(int i=0; i<n; i++) count[(arr[i]/exp)%10]++;
    for(int i=1; i<10; i++) count[i] += count[i-1];
    for(int i=n-1; i>=0; i--) {
        o[count[(arr[i]/exp)%10]--] = arr[i];
    }
    for(int i=0; i<n; i++) arr[i] = o[i];
}

void radixSort(int arr[], int n) {
    int max = getMax(arr, n);
    for(int exp=1; max/exp > 0; exp *= 10)
        countingSort(arr, n, exp);
}

void main() {
    int arr = {329, 457, 657, 839, 436, 720, 355};
    radixSort(arr, 7);
    for(int i=0; i<7; i++) printf("%d ", arr[i]);
}
```

o/p 329 355 436 457 657 720 839
Time = $O(nk)$

M9
15/4

LAB PROGRAM

→ Fractional Knapsack

```

Code #include <stdio.h>
float knapsack_greedy(int p[], int w[], int m, int n)
{
    float x[n], max_profit = 0.0;
    for (int i = 0; i < n; i++)
        x[i] = 0.0;

    int rem_cap = m, i = 0;
    while (rem_cap > 0)
    {
        if (w[i] > rem_cap) break;
        x[i] = 1.0;
        rem_cap = rem_cap - w[i];
        i++;
    }

    if (rem_cap != 0 && i < n) x[i] = (float) rem_cap / w[i];
    for (int i = 0; i < n; i++)
        max_profit = max_profit + (x[i] * p[i]);
    return max_profit;
}

void main()
{
    int n, m;
    printf("Enter number of objects: ");
    scanf("%d", &n);
    printf("Enter max capacity of Knapsack: ");
    scanf("%d", &m);
    int p[n], w[n];
    float wp[n], profit;
    printf("\nEnter profit, weight for %d objects\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d %d", &p[i], &w[i]);
        wp[i] = (float) p[i] / w[i];
    }
}

```



```

for (int i=0; i<n; i++) {
    for (int j=i+1; j<n; j++) {
        if (wp[i] < wp[j]) {
            float temp = wp[i];
            wp[i] = wp[j];
            wp[j] = temp;

```

```

            int temp2 = w[i];
            w[i] = w[j];
            w[j] = temp2;

```

```

            int temp3 = p[i];
            p[i] = p[j];
            p[j] = temp3;
        }
    }
}

```

```

profit = knapsack_greedy(p, w, m, n);
printf("Maximum profit: %f", profit);
}

```

o/p

Enter number of objects: 3

Enter max capacity of Knapsack: 50

Enter profit, weight for 3 objects:

60 10

100 20

120 30

Maximum profit: 240.000

Time Complexity = $O(n^2)$

LAB PROGRAM

Date / /
Page

→ Dijkstra's Algorithm

```

CODE #include <stdio.h>
int main()
{
    int c[100][100], s[100][100], dist[100], v, n;
    printf("Enter the number of vertices");
    scanf("%d", &n);
    printf("Enter cost between each vertex pair");
    for(int i=0; i<n; i++)
    {
        for(int j=0; j<n; j++)
        {
            scanf("%d", &c[i][j]);
        }
    }

    printf("Enter source vertex");
    scanf("%d", &v);
    for(int i=0; i<n; i++)
    {
        s[i] = 0;
        dist[i] = c[v][i];
    }

    s[v] = 1; dist[v] = 0;
    for(int w=0; w<n-1; w++)
    {
        int min = 9999, u;
        for(int i=0; i<n; i++)
        {
            if(s[i] == 0 && dist[i] < min)
            {
                min = dist[i];
                u = i;
            }
        }
        s[u] = 1;
        for(int k=0; k<n; k++)
        {
            if(s[k] == 0 && dist[k] > dist[u] + c[u][k])
            {
                dist[k] = dist[u] + c[u][k];
            }
        }
    }
}

```

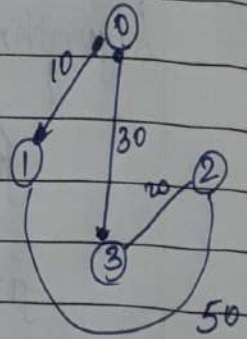
```

for (int i=0; i<n; i++){
    printf("dist[%d] = %d\n", i, dist[i]);
}
return 0;
}

```

o/p Enter number of vertices: 4
cost matrix

0	10	999	30
10	0	50	999
999	50	0	20
30	999	20	0



Enter source vertex: 0

dist[0] = 0

dist[1] = 10

dist[2] = 50

dist[3] = 30

Mg
27/4

LAB PROGRAM

o/1 Knapsack Problem using Dynamic Programming

$$\text{Relation } \text{Table}[i, j] = \begin{cases} 0, & \text{if } i=0 \\ 0, & \text{if } j=0 \\ \text{Table}[i-1, j], & j < w_i \\ \max(\text{Table}[i-1, j], P_i + \text{Table}[i-1, j-w_i]), & \text{if } j \geq w_i \end{cases}$$

Algorithm, Knapsack(n, m, w[], p[])

```

    for i = 0 to n
        Table[i, 0] = 0
    for j = 0 to M
        Table[0, j] = 0
    for i = 0 to n
        for j = 0 to M
            {
                if (j < w[i])
                    then
                        Table[i, j] = Table[i-1, j]
                else if (j >= w[i]) then
                    Table[i, j] = Max(Table[i-1, j], P[i] + Table[i-1, j-w[i]])
            }
        }
    return Table[n, M]

```

Time Complexity:- $O(n \times w)$

```
#include <stdio.h>

Code int max (int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}

int KnapsackDP (int n, int M, int W[], int P[]) {
    int Table[n+1][M+1];
    for (int i=0; i<=n; i++) Table[i][0] = 0;
    for (int j=0; j<=M; j++) Table[0][j] = 0;

    for (int i=1; i<=n; i++) {
        for (int j=1; j<=M; j++) {
            if (j < W[i-1]) {
                Table[i][j] = Table[i-1][j];
            }
            else {
                Table[i][j] = max (Table[i-1][j],
                                   P[i-1] + Table[i-1][j - W[i-1]]);
            }
        }
    }

    return Table[n][M];
}

void main() {
    int n, M;
    printf ("Enter Knapsack capacity (M) {
    no. of objects (n) ");
    scanf ("%d %d", &n, &M);
    printf ("Enter weight & profit array values: ");
    int w[n], P[n];
}
```

```

for (int i=0; i<n; i++) {
    scanf("%d", &w[i]);
    scanf("%d", &p[i]);
}
printf("Max Profit : %d\n", KnapackDP(n, m, w, p));
}

```

o/p Enter Knapack capacity (M) and number of objects (n)

7

Enter weights and profits:

5 7

3 4

4 5

1 1

Max Profit: 9

Enter Knapack capacity (M) and number of objects (n)

4

3

Enter weights and profits:

2 16

2 6

1 18

Max Profit: 34

29/19

LeetCode 801: Minimum Swaps to Make Sequence Increase

```
int min(int a, int b) {
    if (a < b) return a;
    return b;
}
```

```
int minSwap(int *nums1, int nums1Size, int *nums2,
             int *nums2Size) {
    int keep = 0, swap = 1;
    for (int i = 1; i < nums1Size; i++) {
        int nextKeep = INT_MAX;
        int nextSwap = INT_MAX;
        if (nums1[i] > nums1[i-1] && nums2[i] >
            nums2[i-1]) {
            nextKeep = keep;
            nextSwap = swap + 1;
        }
        if (nums1[i] > nums2[i-1] && nums2[i] >
            nums1[i-1]) {
            nextSwap = min(nextSwap, keep + 1);
            nextKeep = min(nextKeep, swap);
        }
        keep = nextKeep;
        swap = nextSwap;
    }
    return min(keep, swap);
}
```

Case 1: [1, 3, 5, 4] = nums1
 [1, 2, 3, 7] = nums2

1

LAB PROGRAM - Kruskal's Algorithm

```
Code #include <stdio.h>
#include <stdlib.h>
#define MAX 30
```

Time $O(E \log E)$

```
int parent[MAX];
int find(int i){
    while (parent[i] != i){
        i = parent[i];
    }
    return i;
}
```

```
void unionSets(int i, int j){
    int a = find(i);
    int b = find(j);
    parent[a] = b;
}
```

```
void main(){
    int cost[MAX][MAX], n, min=0, edges=0;
    printf("Enter number of edges");
    scanf("%d", &n);
    printf("Enter the cost adjacency matrix\n\n");
    for (int i=0; i<n; i++){
        for (int j=0; j<n; j++){
            scanf("%d", &cost[i][j]);
        }
    }
```

```
for (int i=0; i<n; i++)
    parent[i] = i;
printf("\n Edges in the Minimum Spanning Tree: \n");
while (edges < n-1){
    int u=-1, v=-1, min=999;
    for (int i=0; i<n; i++){
        for (int j=0; j<n; j++){
            if (find(i) != find(j) && cost[i][j] < min){
                min = cost[i][j];
                u = i;
                v = j;
            }
        }
    }
    unionSets(u, v);
    edges++;
}
```



```

        }
        }
        }
        if (u != -1 && v != -1) {
            unionSet(u, v);
            printf("Edge (%d %d) with cost %d\n",
                u, v, min);
            minc += min;
            edges++;
        }
        else break;
    }
    if (edges != n-1) {
        printf("No Minimum Spanning Tree\nPossible\n");
    }
    else {
        printf("Minimum cost = %d\n", minc);
    }
}

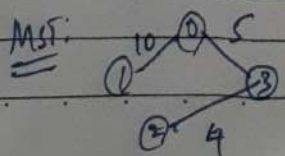
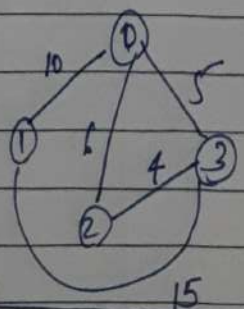
```

o/p

Enter number of vertices: 4
Enter cost adjacency matrix (use 999 for infinity);

0	10	6	5
10	0	999	15
6	999	0	4
5	15	4	0

Edges in Minimum Spanning Tree:
Edge (2,3) with cost 4
Edge (0,3) with cost 5
Edge (0,1) with cost 10
Minimum cost = 19



LAB PROGRAM - Prim's Algorithm

Time

$O(E + V \log V)$

```
#include <stdio.h>
#define INF 999999

int V;

int minkey(int key[], int mstSet[]) {
    int min = INF, min_index = -1;
    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

int primMST (int graph[50][50]) {
    int parent[50], key[50], mstSet[50], mincost = 0;
    for (int i = 0; i < V; i++) {
        key[i] = INF;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V-1; count++) {
        int u = minkey (key, mstSet);
        mstSet[u] = 1;
        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 &&
                graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
}
```

```

    printf("Edge It Weight\n");
    for(int i=1; i<V; i++){
        printf("%d - %d | %d\n",
            parent[i], i, graph[i][parent[i]]);
        mincost += graph[i][parent[i]];
    }
    return mincost;
}

int main(){
    int graph[50][50];
    printf("Enter number of vertices!!");
    scanf("%d", &V);
    printf("Enter adjacency matrix (use 0 if no edge):\n");
    for(int i=0; i<V; i++){
        for(int j=0; j<V; j++){
            scanf("%d", &graph[i][j]);
        }
    }
    int cost = printMST(graph);
    printf("Minimum cost of MST = %d\n", cost);
    return 0;
}

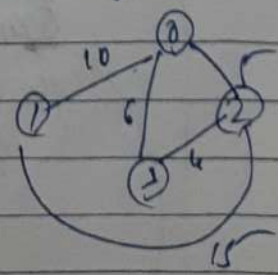
```

o/p

Enter number of vertices: 4
 Enter adjacency matrix (use 0 if no edge):

0	10	6	5
10	0	0	15
6	0	0	4
5	15	4	0

MH
4/5



Edge Weight

- 0-1 10
- 0-2 6
- 0-3 5
- 1-2 4
- 1-3 15
- 2-3 4

MST



Minimum cost of MST = 24

LAB PROGRAM - Floyd's Algorithm

Time Complexity: $O(n^3)$ Algorithm

CODE

```

#include <stdio.h>
#define INF 99
void floyd(int n, int cost[10][10], int A[10][10])
{
    int i, j, k;
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            A[i][j] = cost[i][j];
    for (k = 1; k <= n; k++)
        for (i = 1; i <= n; i++)
            for (j = 1; j <= n; j++)
                if (A[i][k] + A[k][j] < A[i][j])
                    A[i][j] = A[i][k] + A[k][j];
}

void main()
{
    int n, i, j;
    int cost[10][10], A[10][10];
    printf("Enter number of vertices:");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            scanf("%d", &cost[i][j]);

    floyd(n, cost, A);

    printf("Enter the cost matrix (use 99 for infinity):");
}

```



```

floyd (n, cost, A);
printf ("\n Shortest Path: \n");
for (i=1; i<=n; i++) {
    for (j=1; j<=n; j++) {
        if (A[i][j] == INF)
            printf ("INF");
        else
            printf ("%d", A[i][j]);
    }
    printf ("\n");
}
}

```

o/p

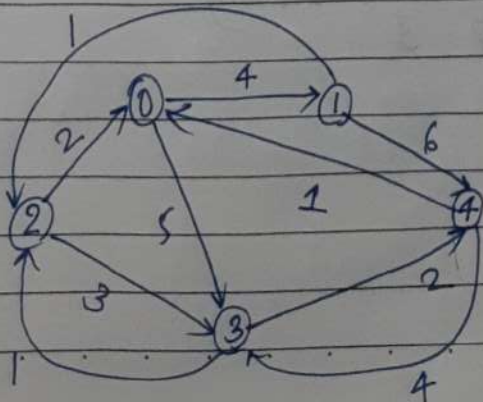
Enter number of vertices: 5

Enter the cost matrix (use 99 for infinity):

0	4	99	5	99
99	0	1	99	6
2	99	0	3	99
99	99	1	0	2
1	99	99	4	0

Shortest Path:

0	4	5	5	7
3	0	1	4	6
2	6	0	3	5
3	7	1	0	2
1	5	5	4	0



LeetCode on Floyd's Algorithm

Date / /
Page

287.

```
int findDuplicate (int * nums, int numsSize) {  
    int s = nums[0];  
    int f = nums[0];  
    do {  
        s = nums[s];  
        f = nums[nums[f]];  
    } while (s != f);  
    while (s != f) {  
        s = nums[s];  
        f = nums[f];  
    }  
    return s;  
}
```

Case 1

nums = [1, 3, 4, 2, 2]
o/p = 2

Case 2

nums = [3, 1, 3, 4, 2]
o/p = 3

Case 3

nums = [3, 3, 3, 3, 3]
o/p = 3

AS PROGRAM: Heap Sort

Date: / /
Page:

```
#include <stdio.h>

void heapify (int a[], int n, int p){
    int c, item;
    item = a[p];
    c = 2*p+1;
    while (c <= n-1){
        if (c+1 <= n-1)
            if (a[c] < a[c+1])
                c++;
        if (item < a[c]){
            a[p] = a[c];
            p = c;
            c = 2*p+1;
        }
        else{
            break;
        }
    }
    a[p] = item;
}

void build_heap (int a[], int n){
    int i;
    for (i = (n-1)/2; i >= 0; i--){
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify (a, i, 0);
        heapify (a, n, i);
        heapify (a, n, i);
    }
}
```



```

void heapSort (int a[], int n) {
    int i, temp;
    buildHeap (a, n);
    for (i = n-1; i > 0; i--) {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify (a, i, 0);
    }
}

```

```

void main() {
    int a[], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter array elements: \n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    heapSort (a, n);
    printf("Sorted Array: \n");
    for (int i = 0; i < n; i++) {
        printf("%d | ");
    }
}

```

Enter number of elements: 5
 Enter array elements:
 23 48 35 99 54
 Sorted array:
 23 35 48 54 99

11/5/26
 11/5/26

Time: $O(n \log n)$

LAB PROGRAM: "N-Queens" using Backtracking

Date 25/05/2024
Page

Algo: Algorithm Nqueens(n) {
 $k=1$; $x[k]=0$;
 while ($k > 0$) {
 $x[k] = x[k] + 1$;
 while ($x[k] \leq n$ and NOT PLACE($k, x[k]$))
 $x[k] = x[k] + 1$;
 if ($x[k] \leq n$)
 {
 if ($k == n$) Print sol.
 else {
 $k++$; $x[k]=0$;
 }
 }
 else {
 $k--$;
 }
 }
}

CODE: #include <stdio.h>

int n;

int x[20];

void nqueens(int n) {

int k=1;

$x[k]=0$;

~~while~~ ($k > 0$) {

$x[k] = x[k] + 1$;

if ($x[k] \leq n$) {

int ok=1;

for (int j=1; j < k; j++) {

if ($x[j] == x[k]$ || $(j-k) == (x[j] - x[k])$
 || $(j+k) == (x[j] + x[k])) {$

ok=0;

break;

```

        if (ok) {
            if (k == n) {
                for (int i = 1; i <= n; i++) {
                    printf("%d", x[i]);
                }
                printf("\n");
            }
            else {
                k++;
                x[k] = 0;
            }
        }
    }
}
else {
    k--;
}
}
}

void main() {
    printf("Enter number of queens: ");
    scanf("%d", &n);
    nqueens(n);
}

```

o/p Enter number of queens: 4
 2 4 1 3
 3 1 4 2

MG
 27/5/26